



Natural Language Processing: Transformers

HSE Faculty of Computer Science

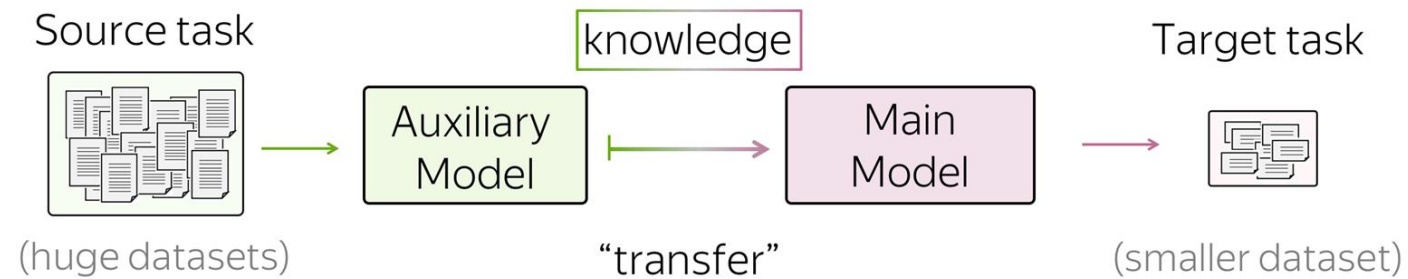
Murat Khazhgeriev



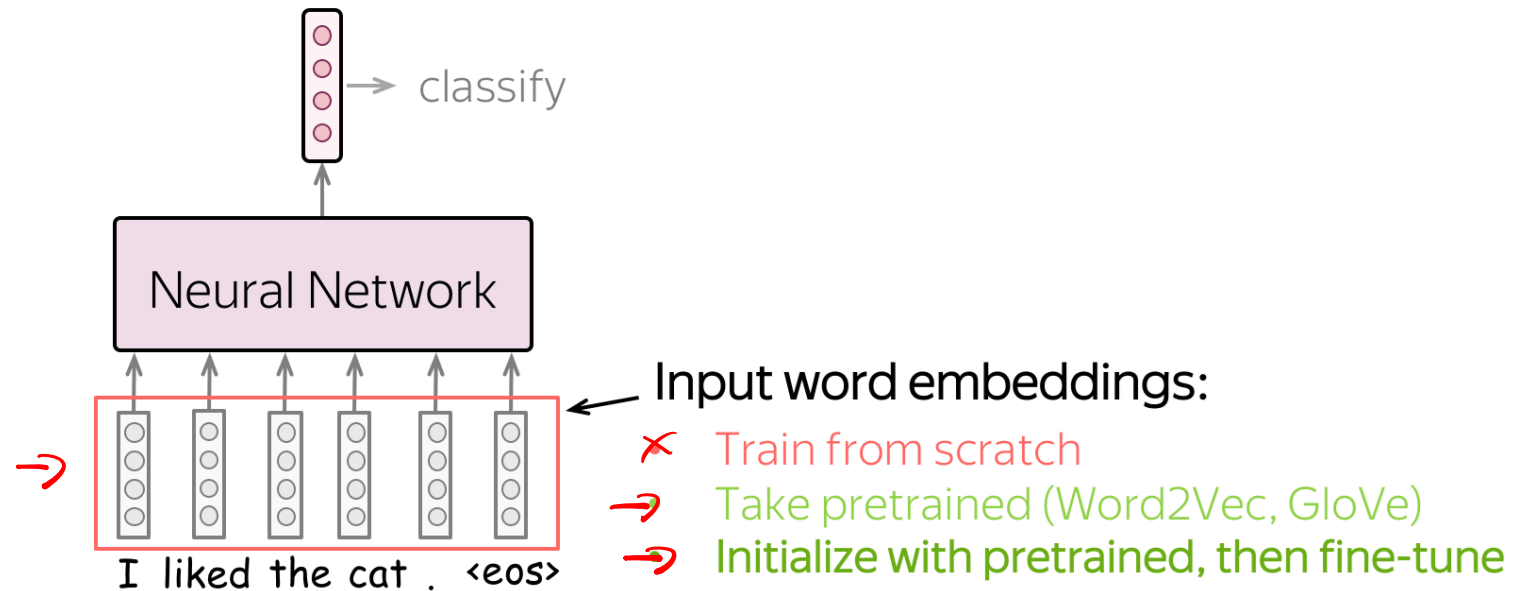
Table of Content

- **The power of transfer learning**
- From word-specific to contextual embeddings
- Transformer architecture overview
- BERT
- GPT

Training a model on a general task can benefit a downstream one



Training a model on a general task can benefit a downstream one



Training a model on a general task can benefit a downstream one

- Train from scratch

What they will know:



May be not enough to learn relationships between words

- Take pretrained (Word2Vec, GloVe)

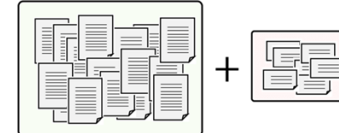
What they will know:



Know relationships between words, but are **not** specific to the task

- Initialize with pretrained, then fine-tune

What they will know:



Know relationships between words and adapted for the task

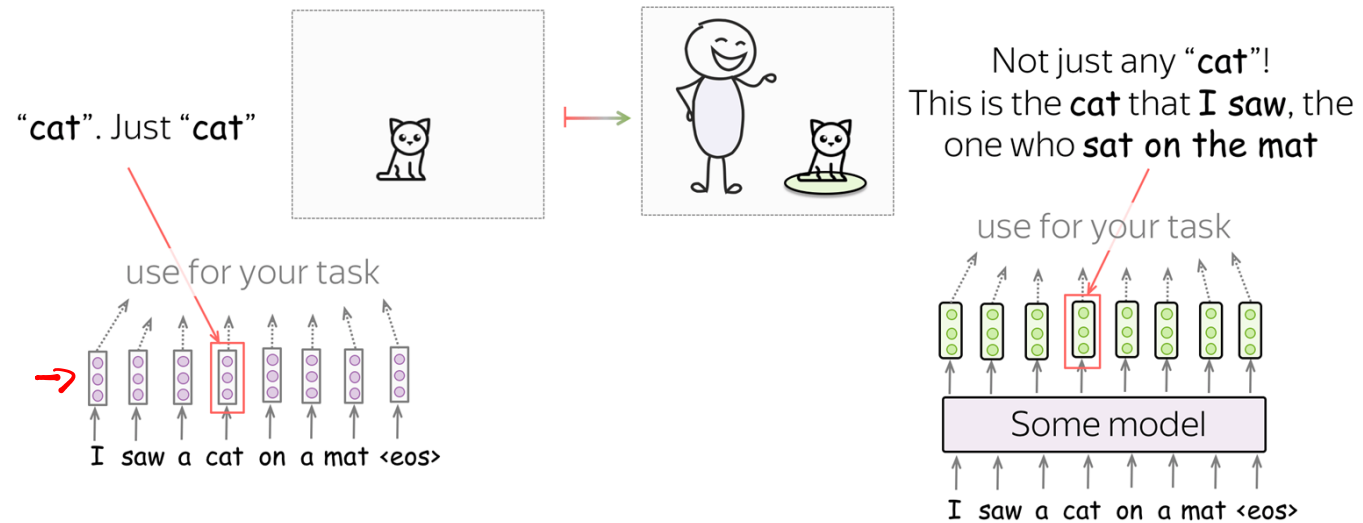
“Transfer” knowledge from a huge unlabeled corpus to your task-specific model



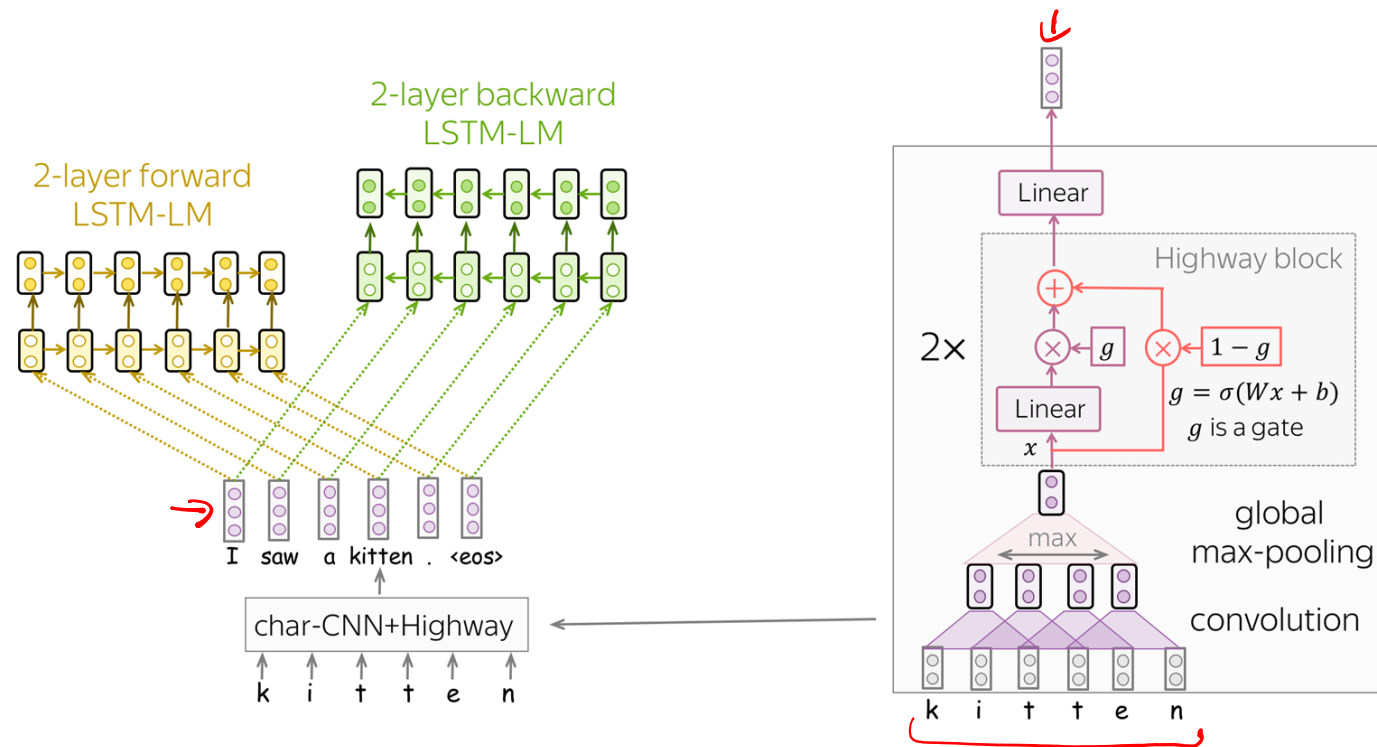
Table of Content

- The power of transfer learning
- **From word-specific to contextual embeddings**
- Transformer architecture overview
- BERT
- GPT

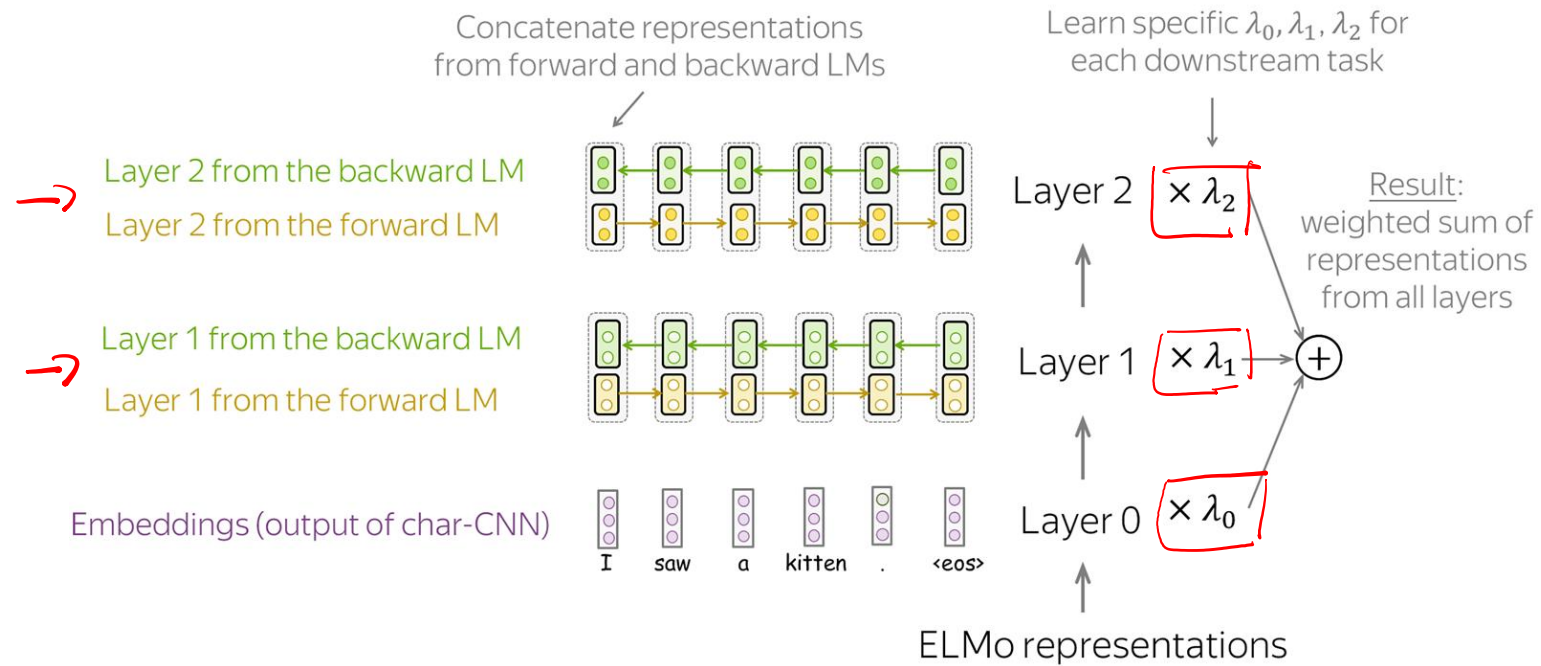
Not just a cat, but the cat!



Train a “translator” from word-specific to “contextual” space



Multiple layers to capture low-level and high-level context



From an embedding generator to a universal model

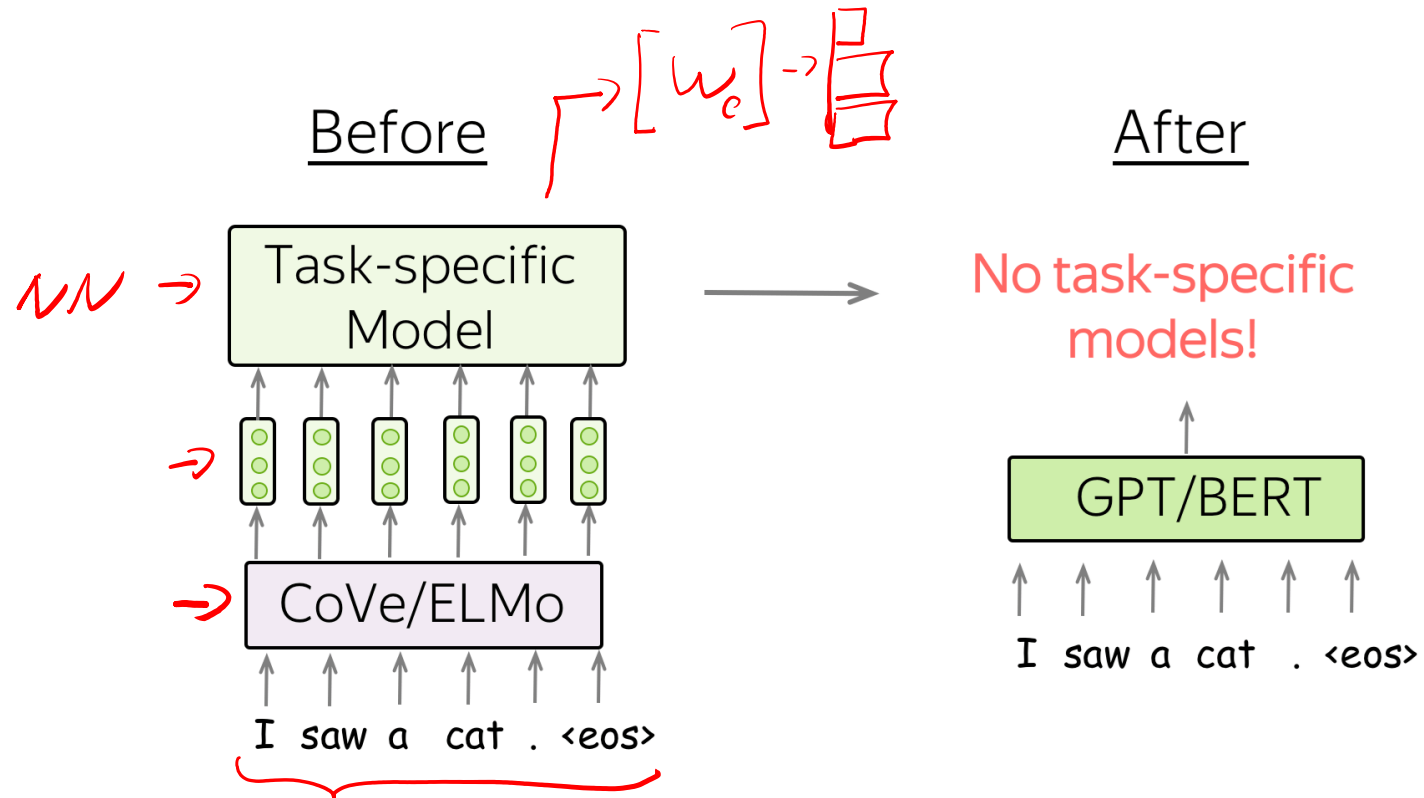
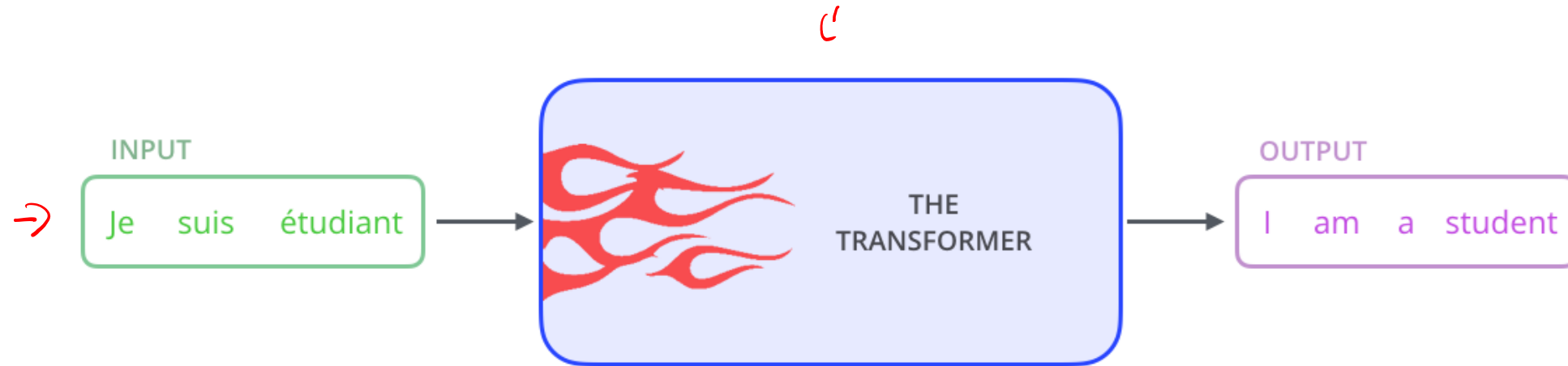




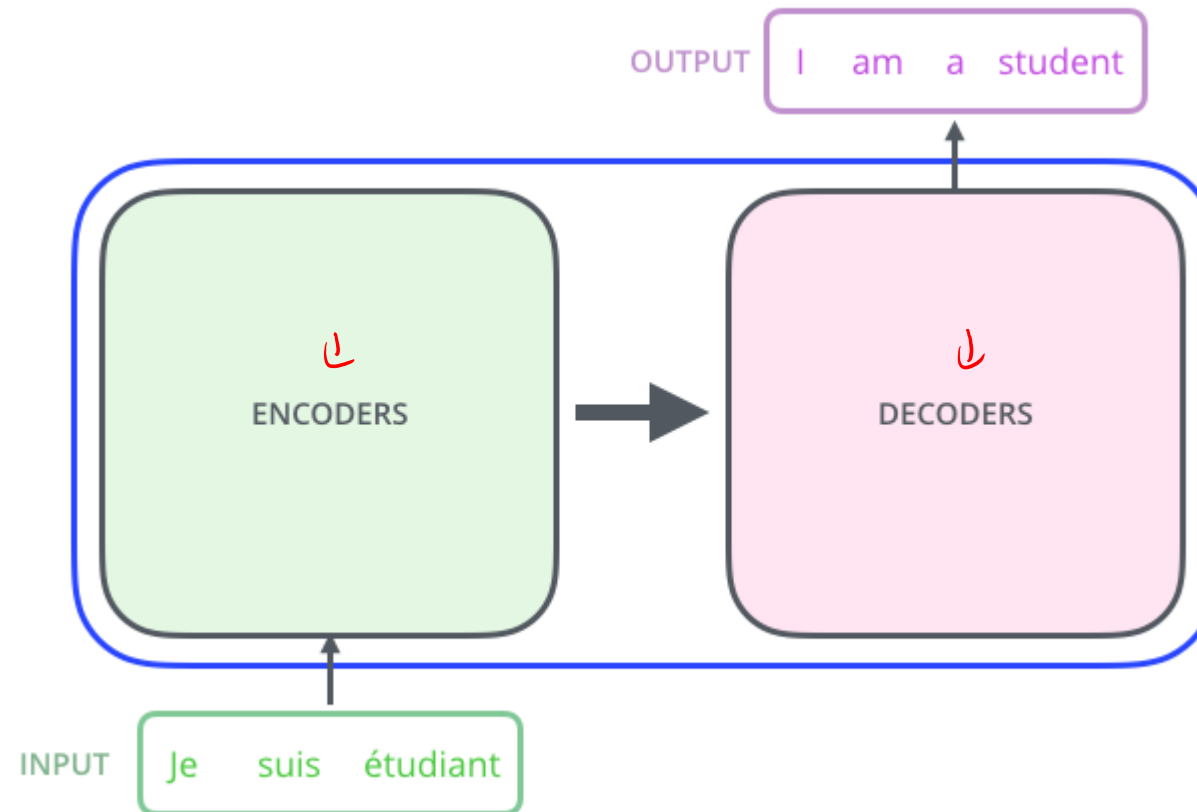
Table of Content

- The power of transfer learning
- From word-specific to contextual embeddings
- **Transformer architecture overview**
- BERT
- GPT

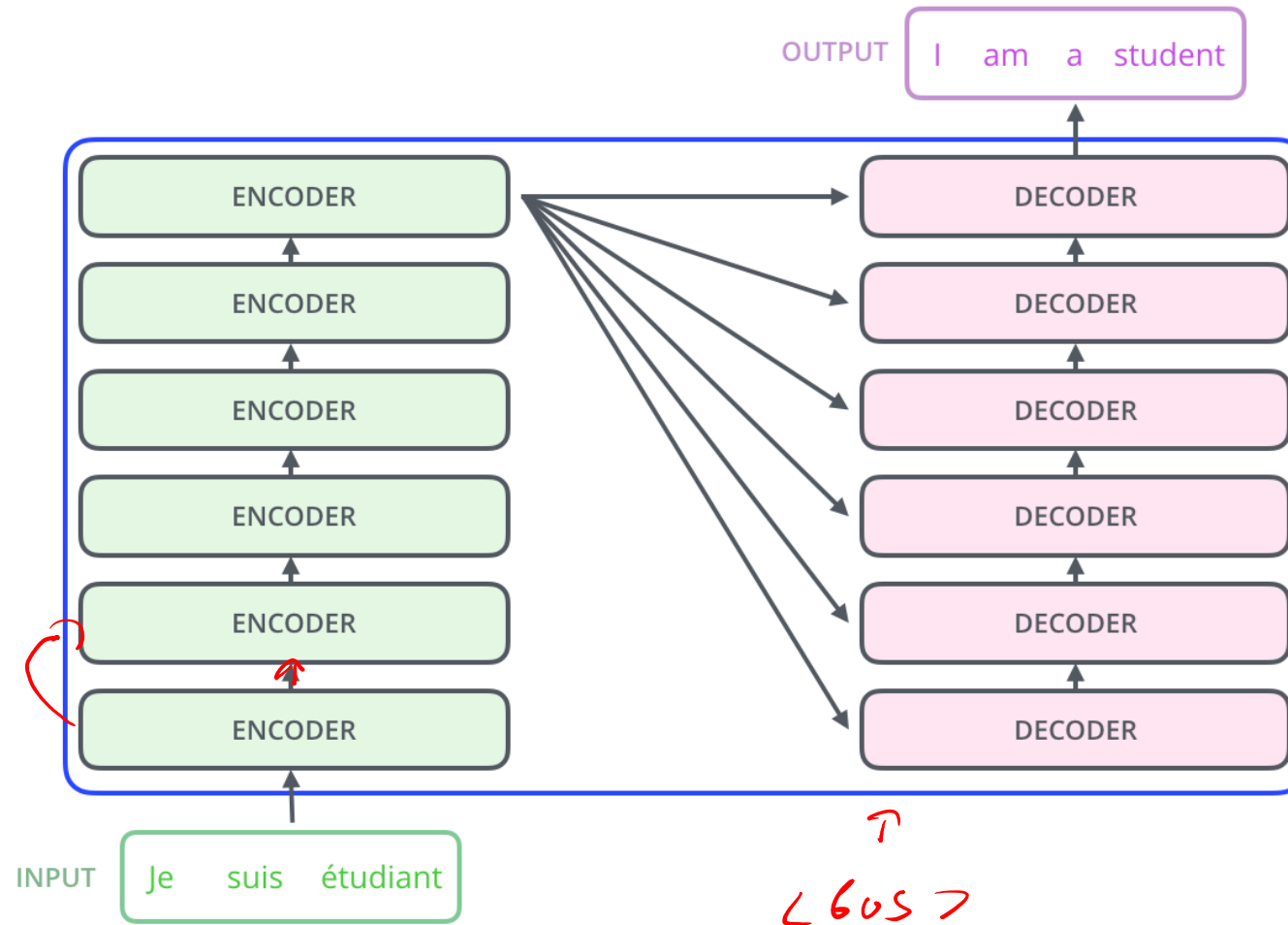
Transformer is an example of Encoder-Decoder architecture



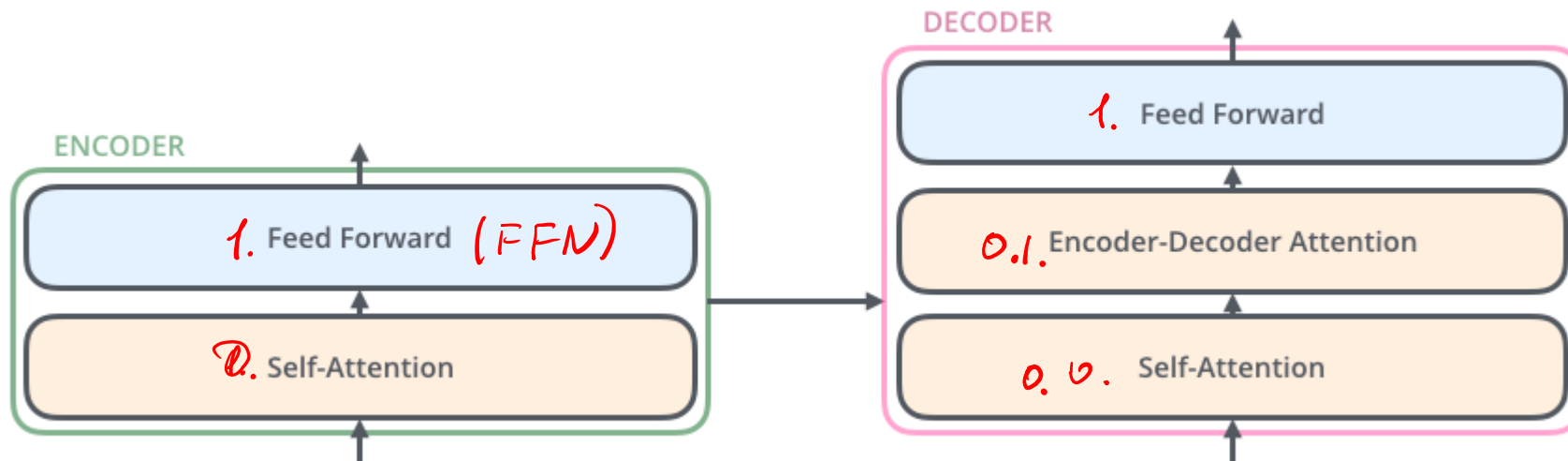
Transformer is an example of Encoder-Decoder architecture



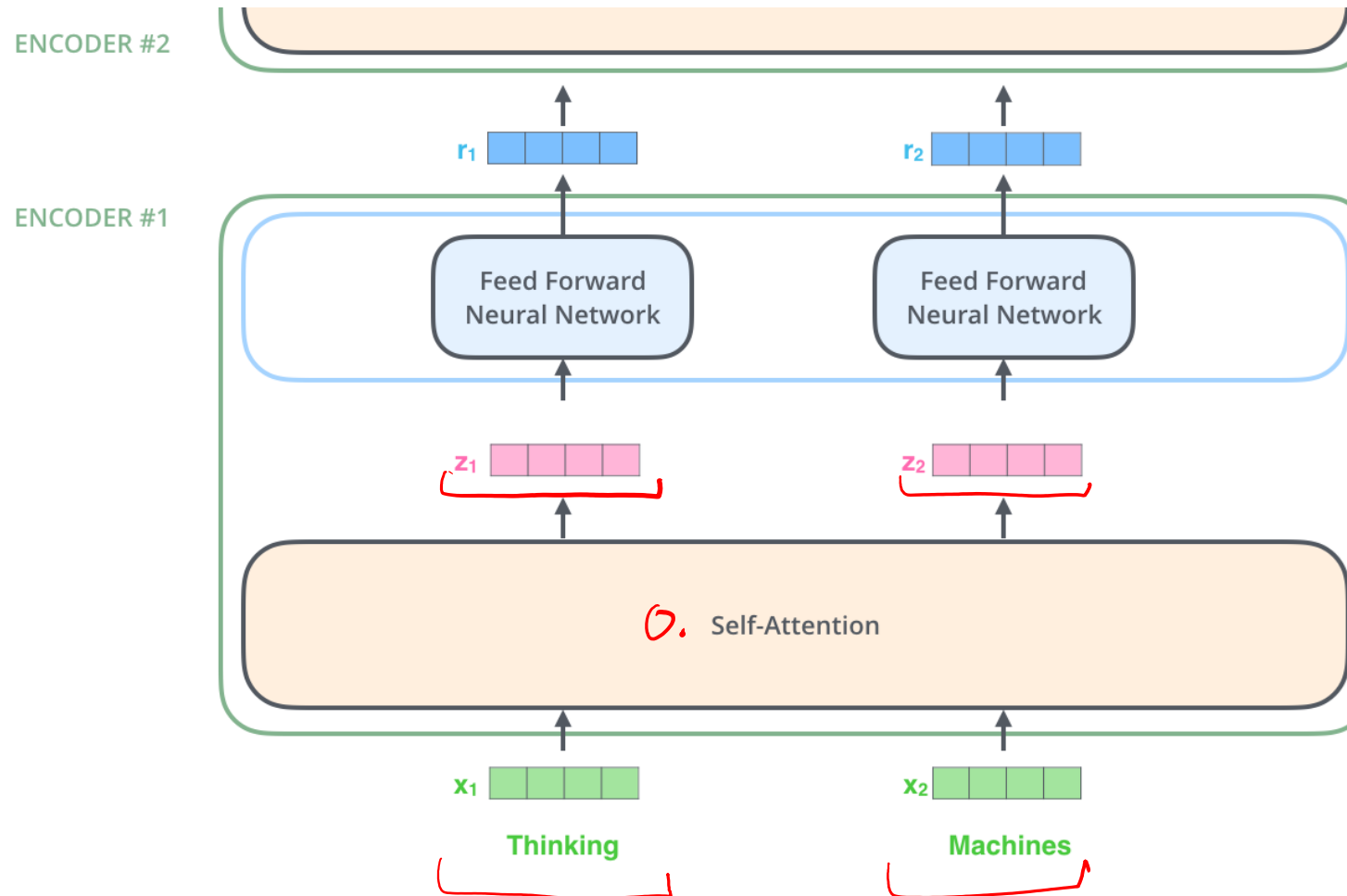
Transformer is an example of Encoder-Decoder architecture



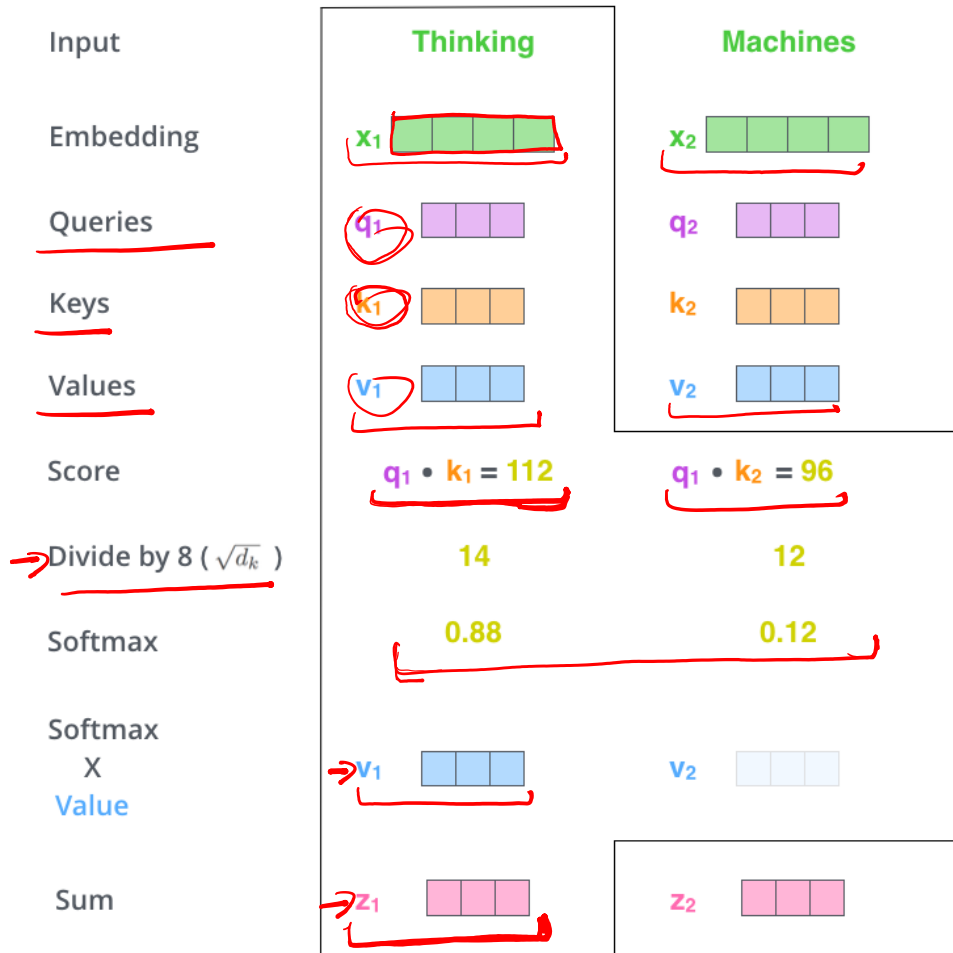
Always two there are. No more, no less. An Attention and an FFN. © Yoda



Always two there are. No more, no less. An Attention and an FFN. © Yoda



Inside a Self-Attention



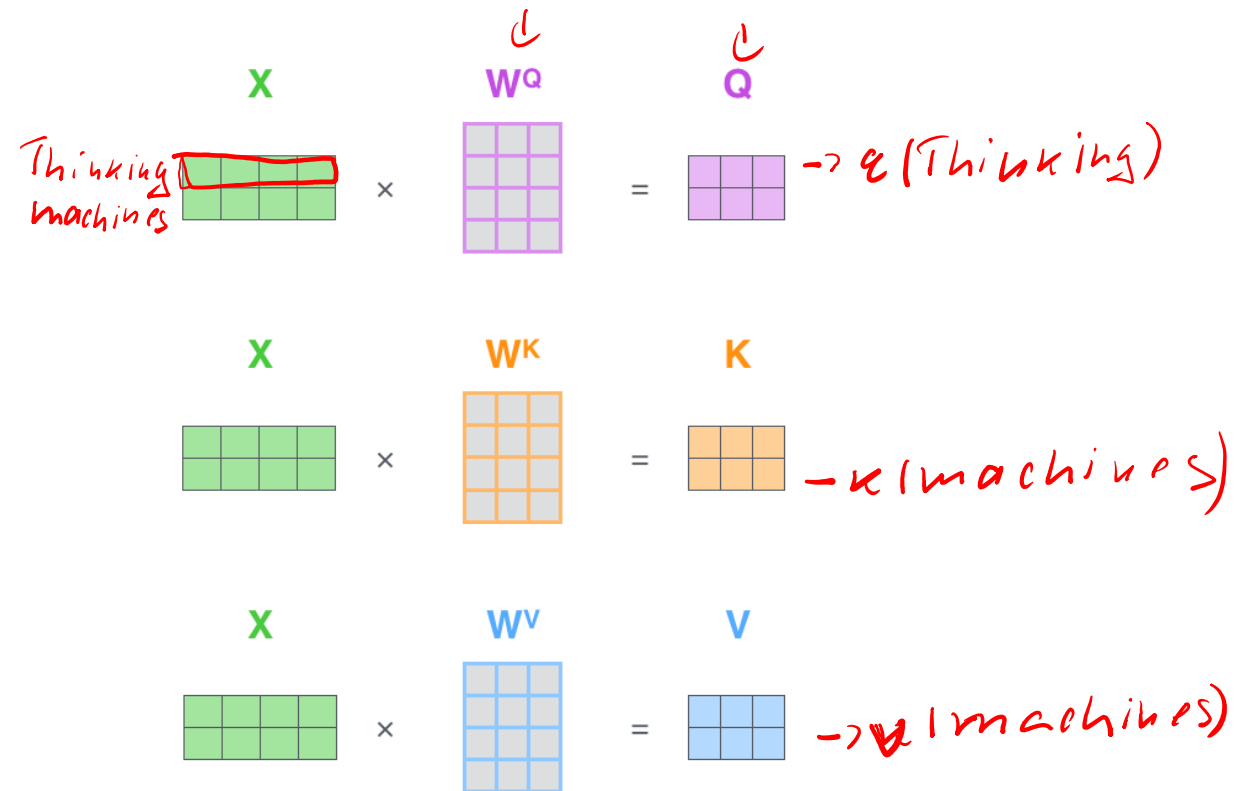
Handwritten notes:

- $w_q \rightarrow q$
- $x_1 \rightarrow k$
- $w_k \rightarrow k$
- $w_v \rightarrow v$

Handwritten notes:

- A vector $\begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$ is shown with a softmax operation applied to it, resulting in a probability distribution over three categories.

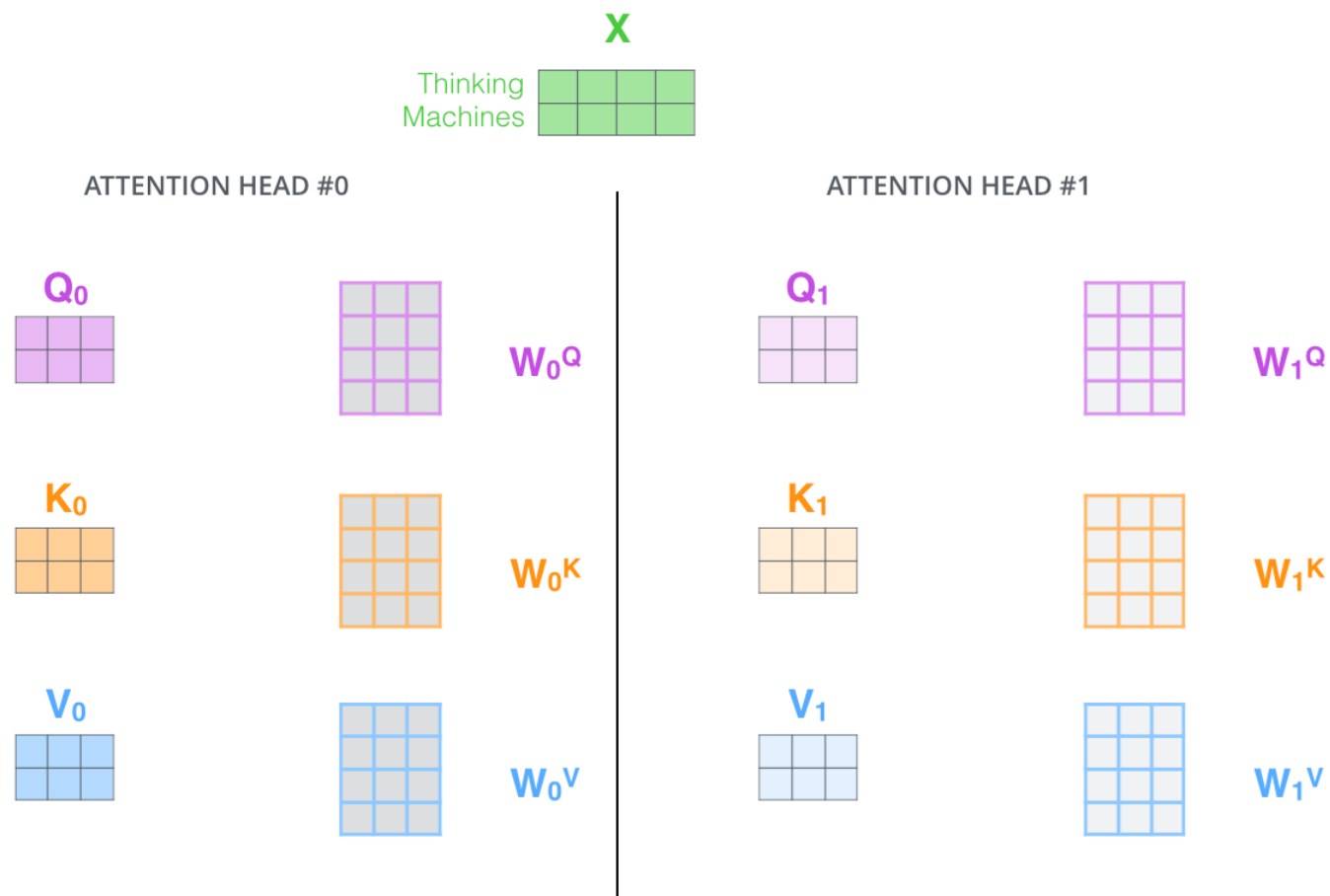
Inside a Self-Attention: Matrix View



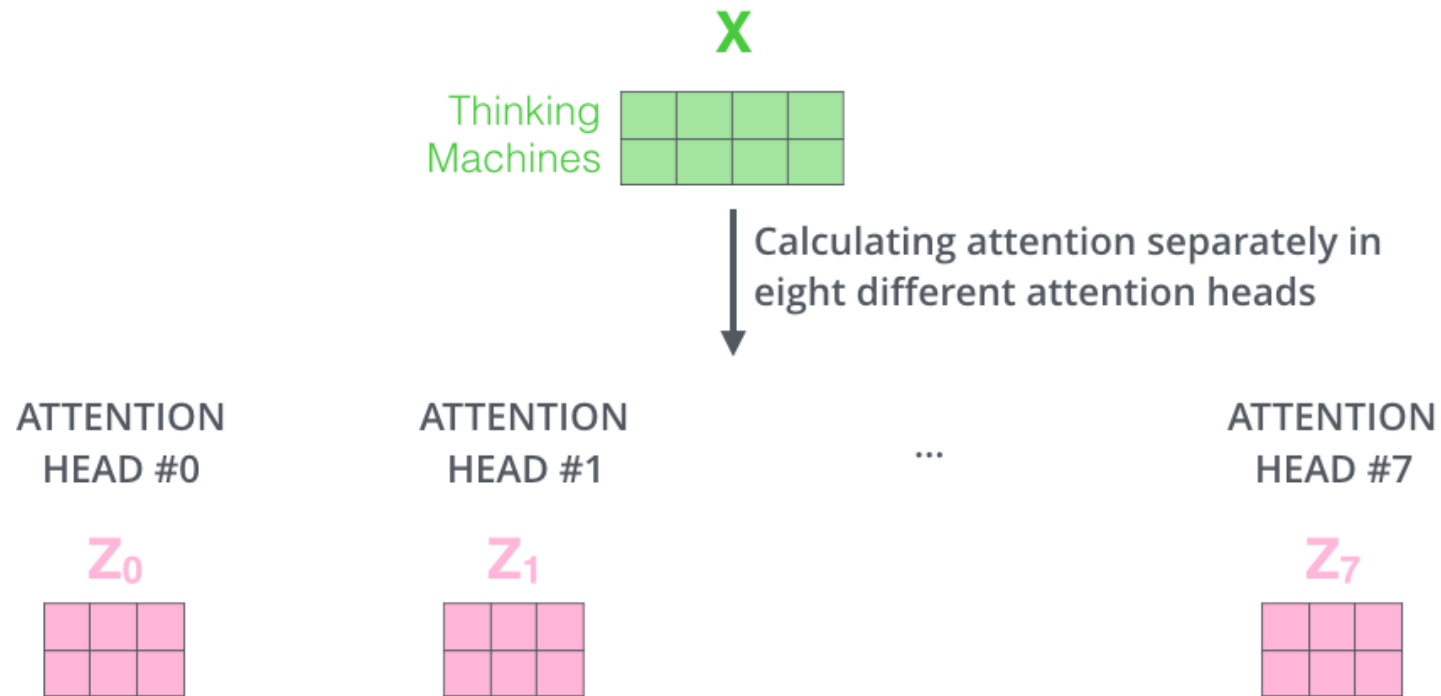
Inside a Self-Attention: Matrix View

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$
$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

A beast with many heads

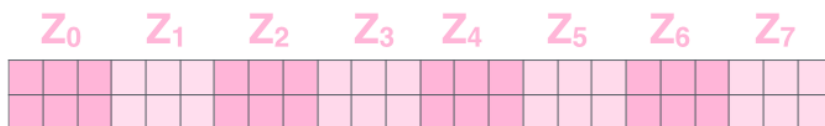


A beast with many heads



A beast with many heads

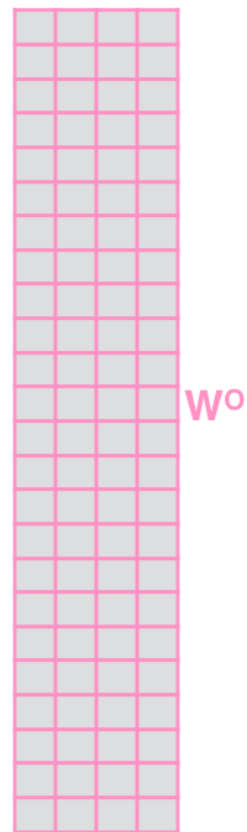
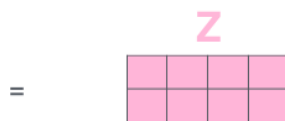
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

X

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

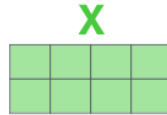


A multi-head attention overview

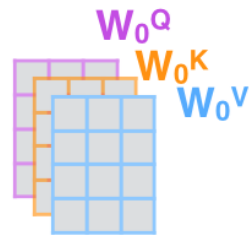
1) This is our
input sentence*

Thinking
Machines

2) We embed
each word*



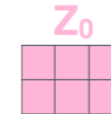
3) Split into 8 heads.
We multiply X or
 R with weight matrices



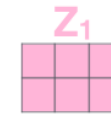
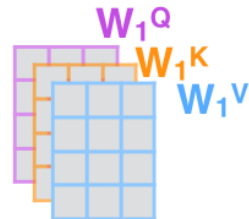
4) Calculate attention
using the resulting
 $Q/K/V$ matrices



5) Concatenate the resulting Z matrices,
then multiply with weight matrix W^O to
produce the output of the layer



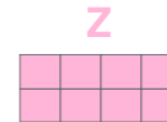
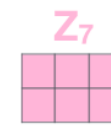
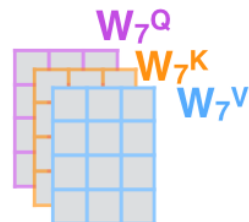
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



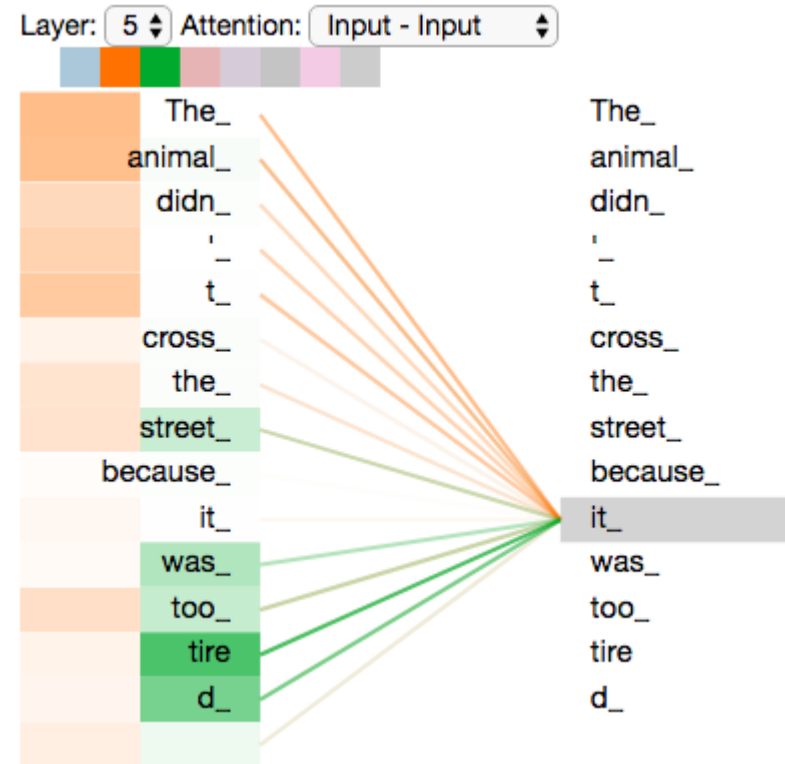
...

...

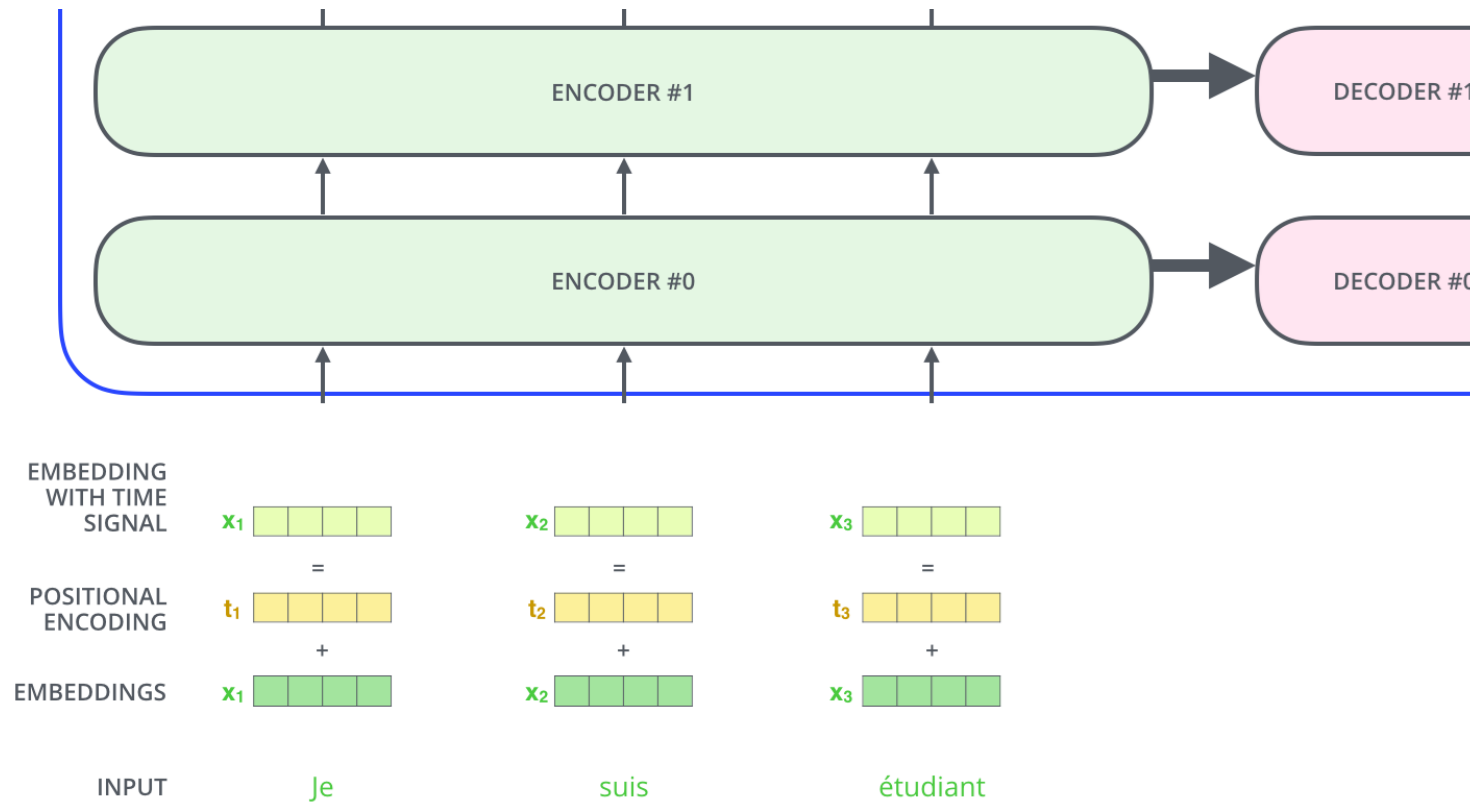
...



Each head focuses on a specific representation



As opposed to RNNs, Transformers do not track the position implicitly

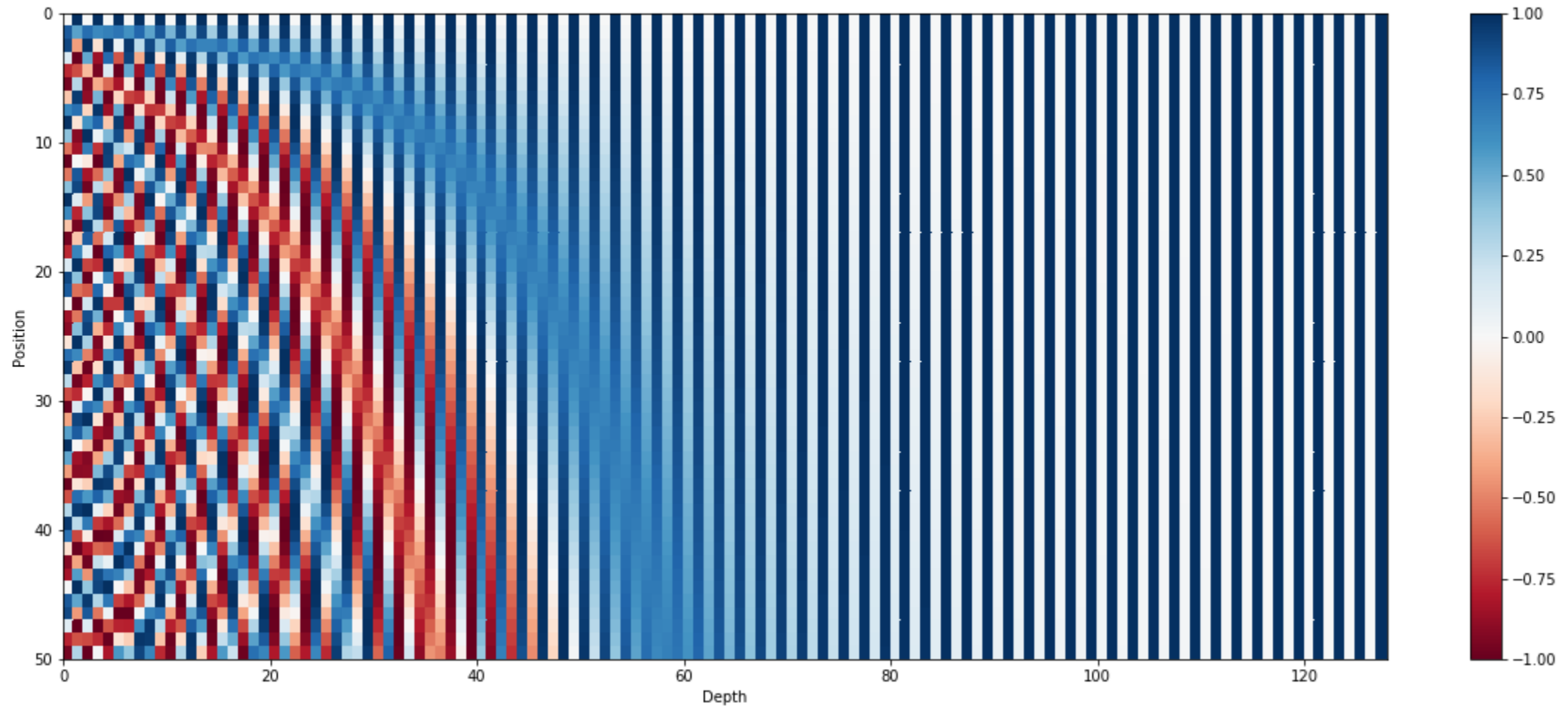


Absolute Positional Encoding

$$\text{PE}(\text{pos}, 2i) = \sin\left(\text{pos}/10000^{2i/d_{\text{model}}}\right)$$

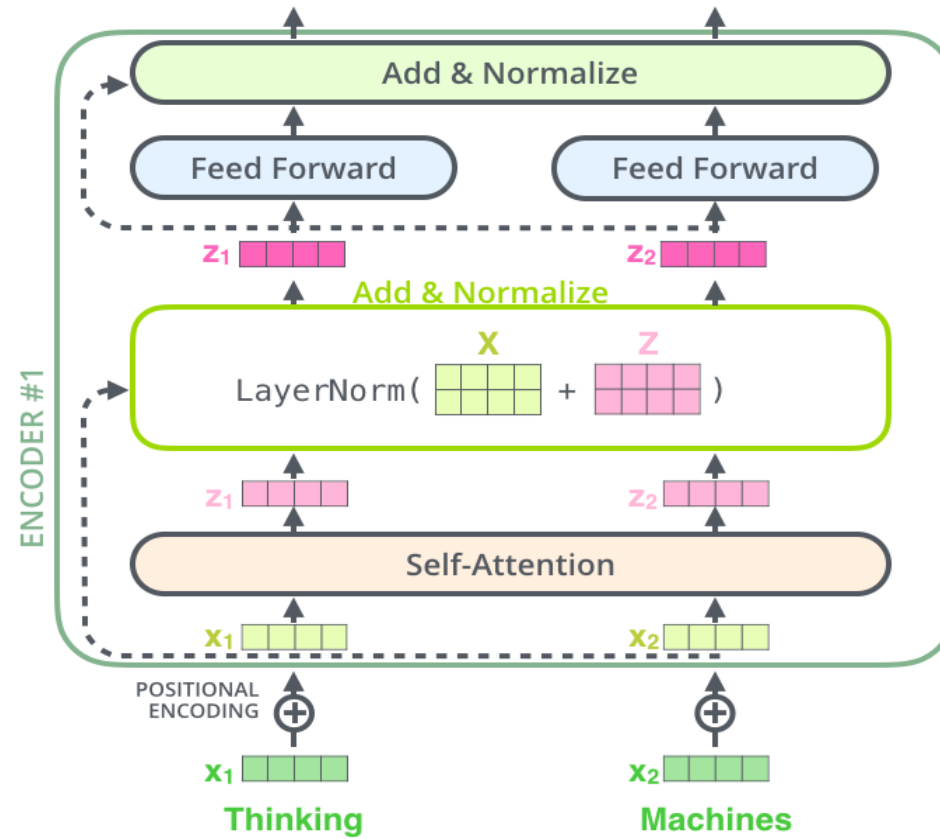
$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\text{pos}/10000^{2i/d_{\text{model}}}\right)$$

What it looks like

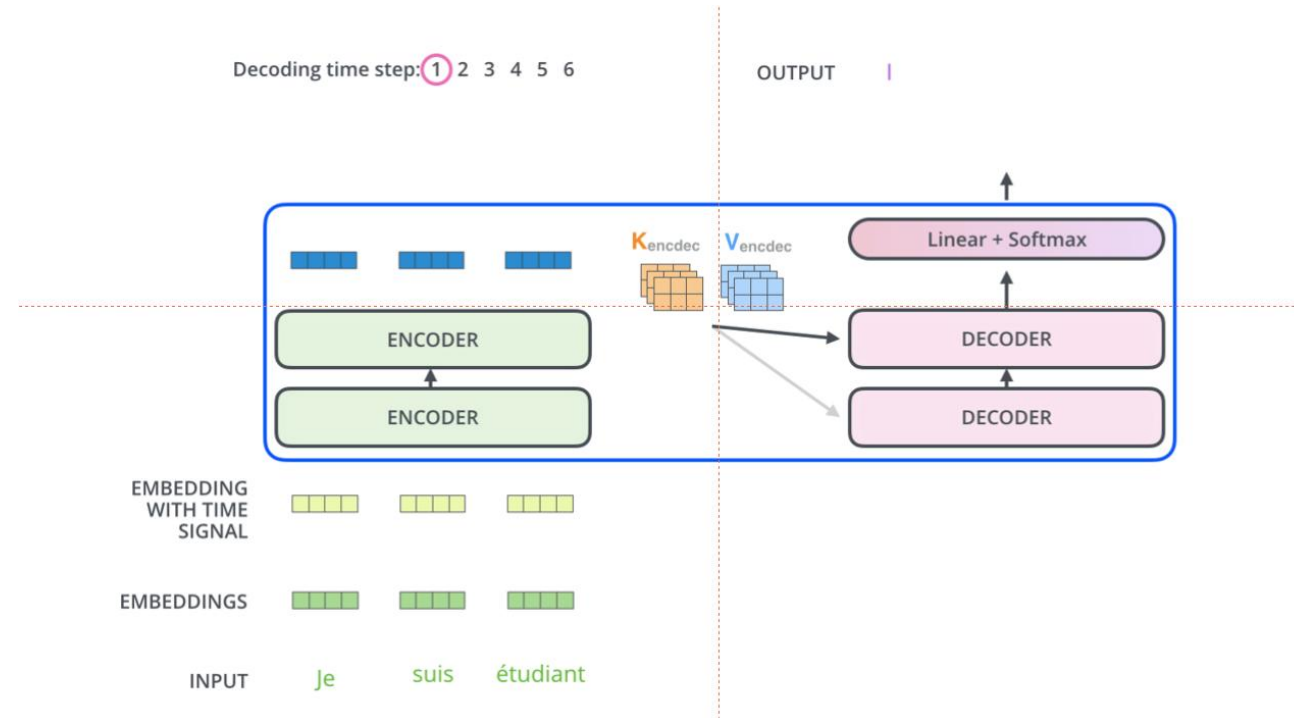


Source: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

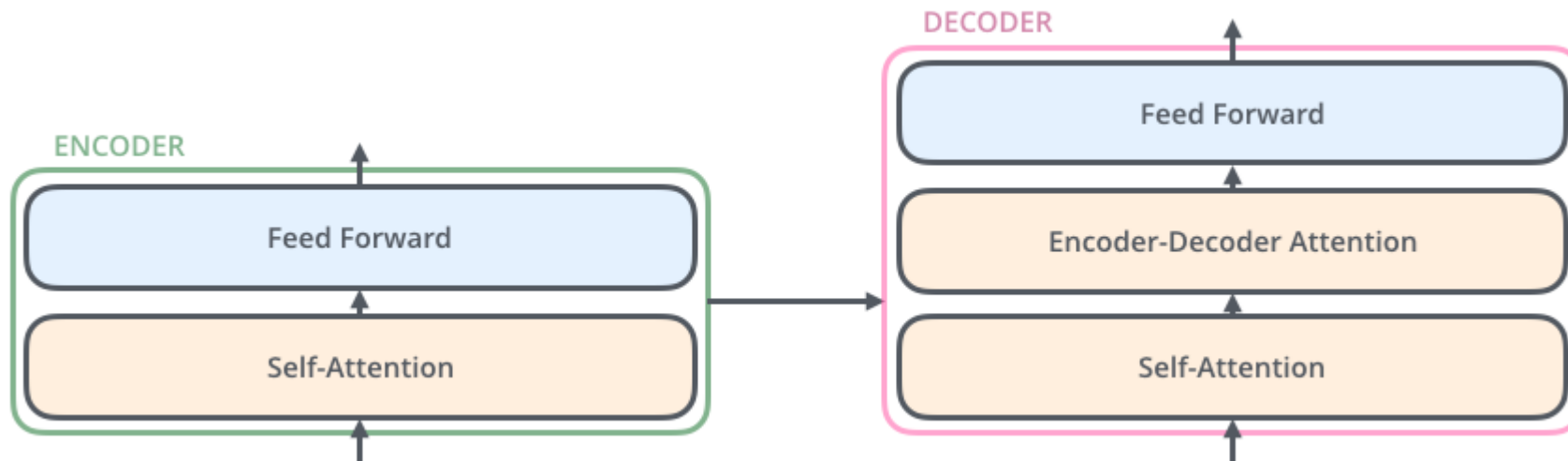
Skip Connection and Layer Normalization for a robust training



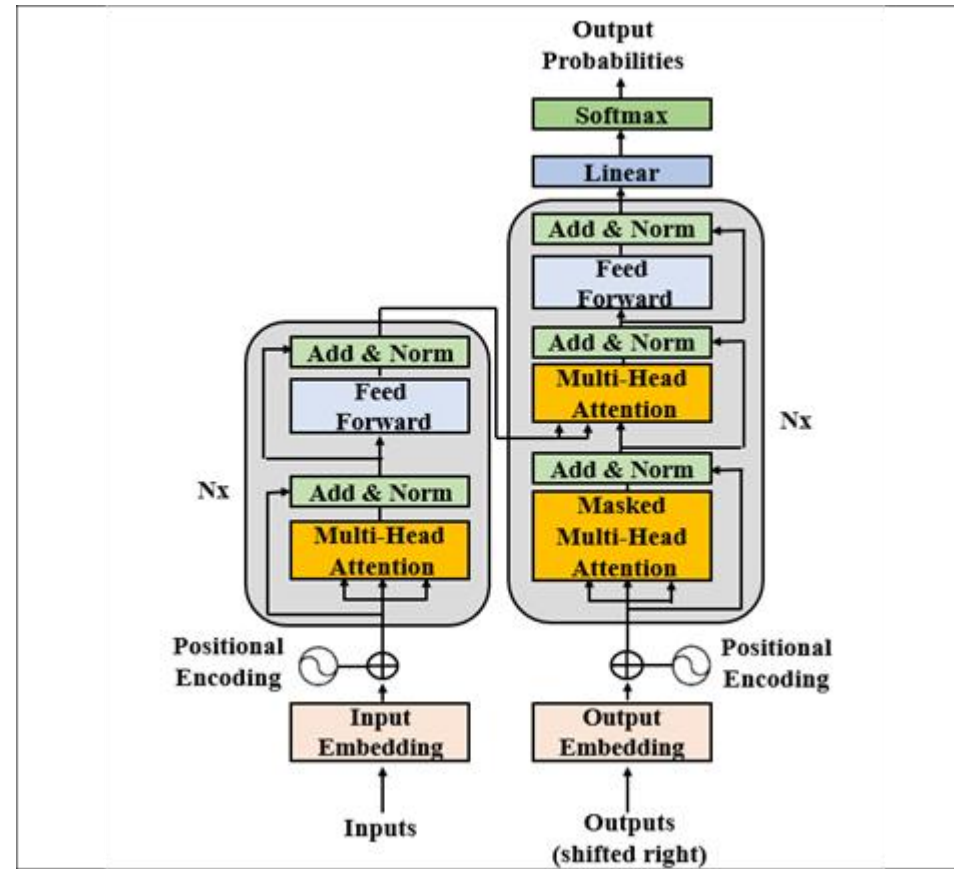
Bringing it all together



Always two there are. No more, no less. An Attention and an FFN. © Yoda



Attention is all you need



Attention is all you need (but not really)

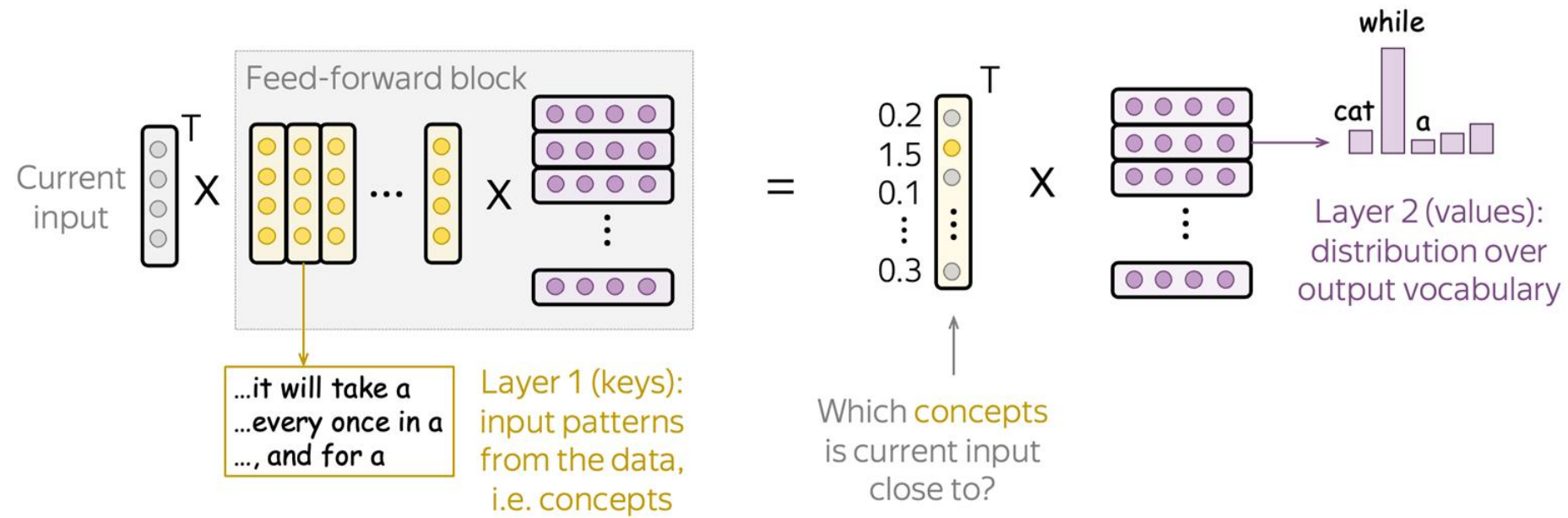
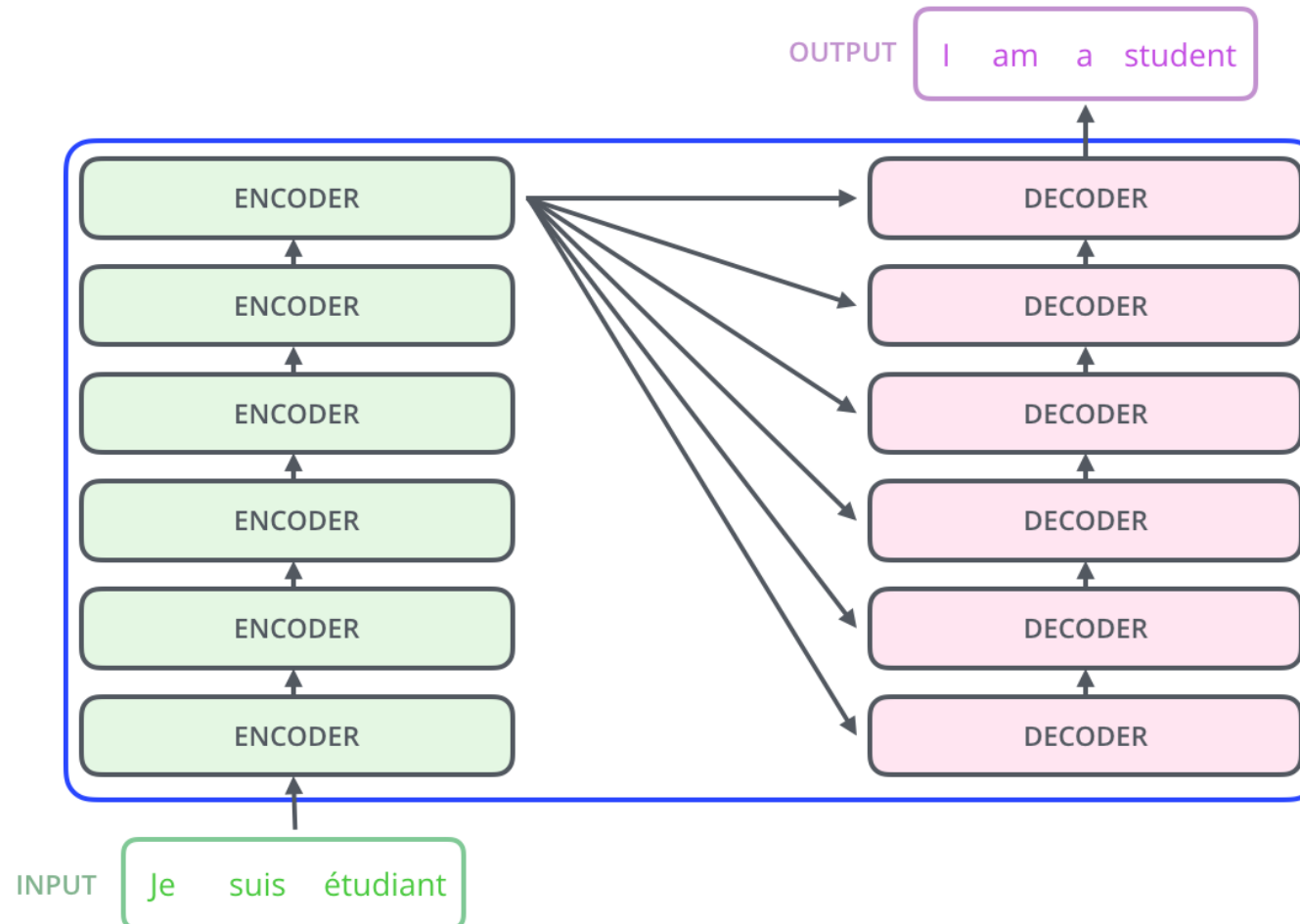




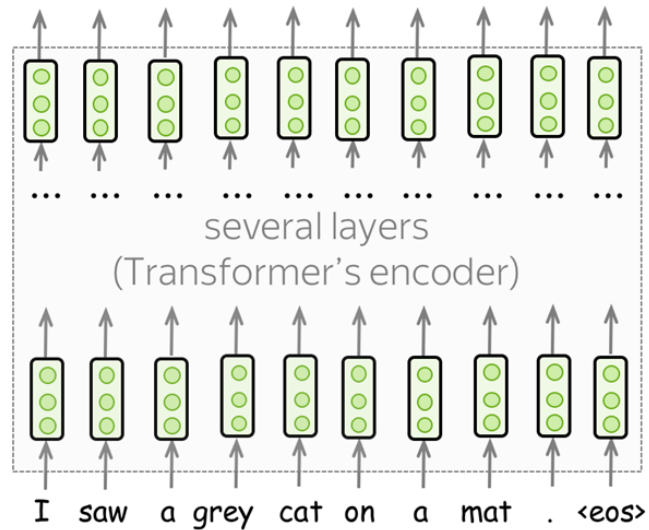
Table of Content

- The power of transfer learning
- From word-specific to contextual embeddings
- Transformer architecture overview
- **BERT**
- GPT

BERT is just an Encoder part



Using encoder as an embedding generator



Model architecture:

- Transformer's encoder

What is special about it:

- Training objectives
 - MLM: Masked language modeling
 - NSP: Next sentence prediction
- The way it is used
 - No task-specific models

Objective one: tell whether the two sequences are consecutive

[CLS]: Special token

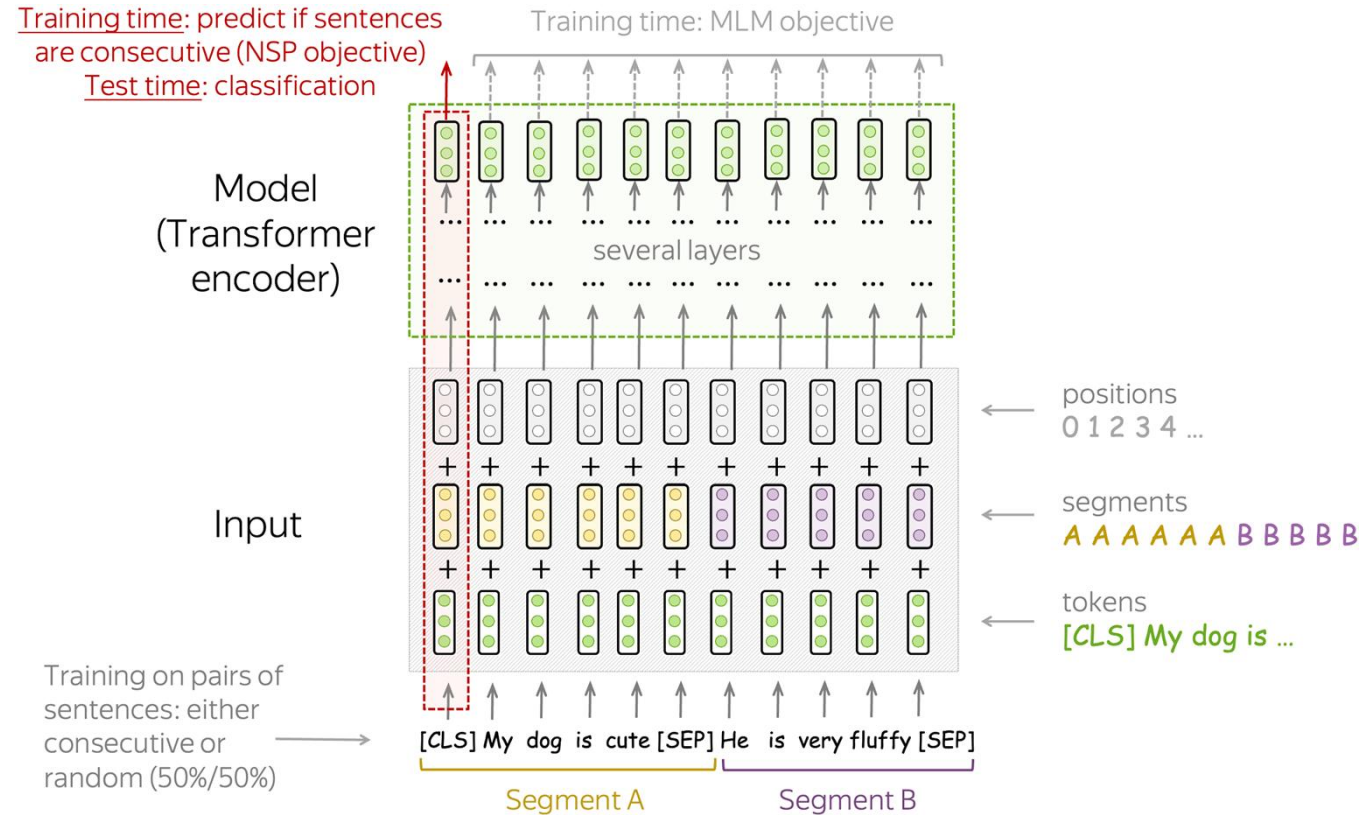
- Training time: predict if sentences are consecutive or not (Next Sentence Prediction /NSP objective)
- Test time: downstream tasks (e.g., classification)

[SEP]: Special token-separator

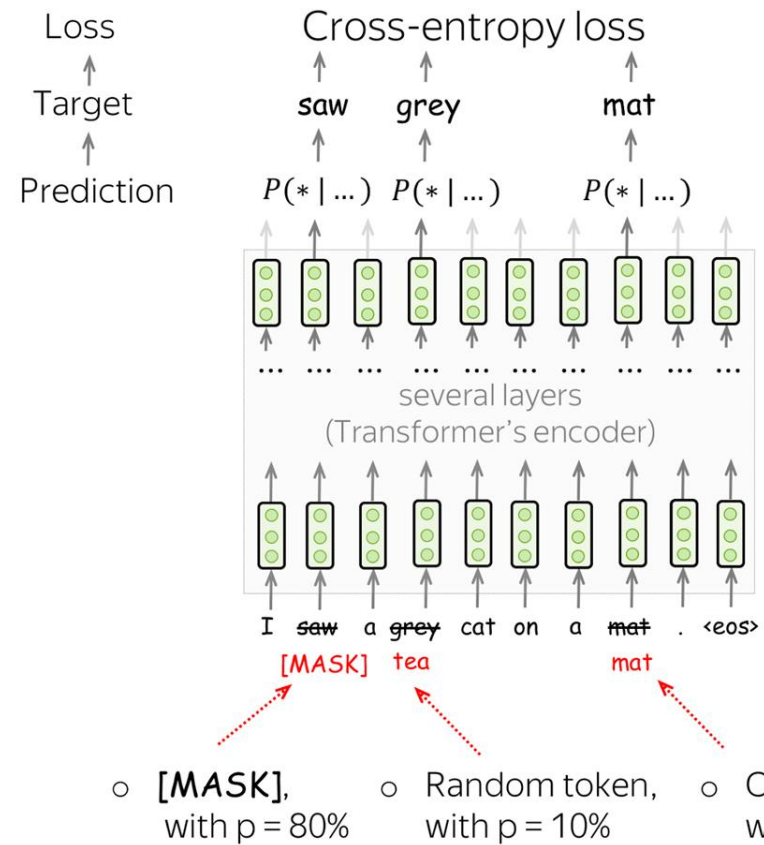


Training on pairs of sentences: either consecutive or random (50%/50%)

Using encoder as an embedding generator



Objective two: Masked Language Modeling



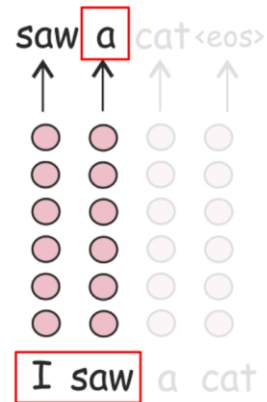
At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

LM vs. MLM

Language Modeling

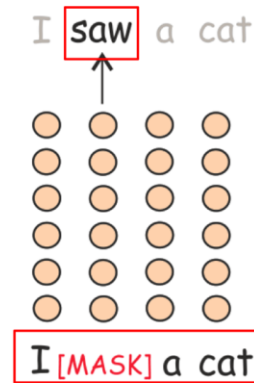
- Target: next token
- Prediction: $P(* | \mathbf{I} \text{ saw})$



left-to-right, does
not see future

Masked Language Modeling

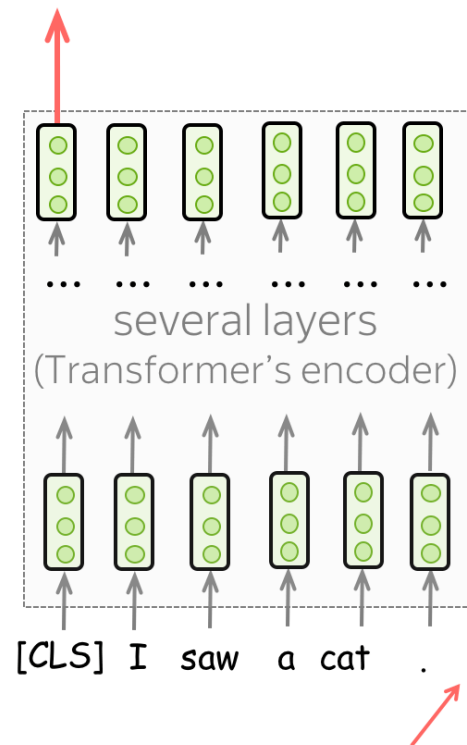
- Target: current token (the true one)
- Prediction: $P(* | \mathbf{I} [\text{MASK}] \text{ a cat})$



sees the whole text, but
something is corrupted

Single Sentence Classification

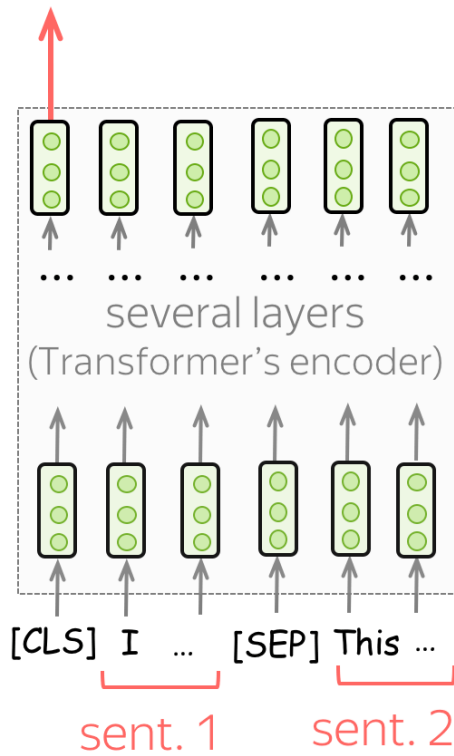
class label



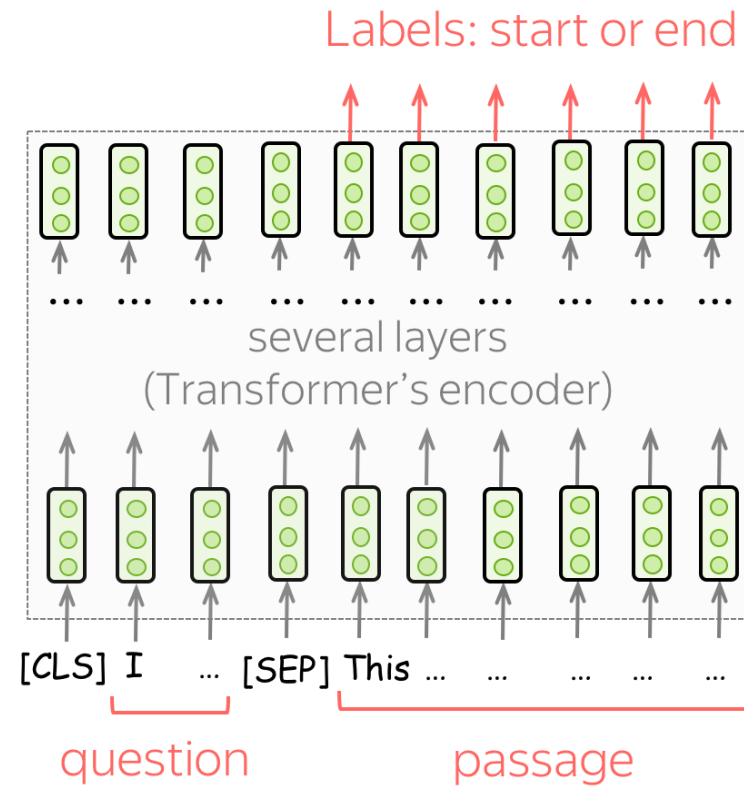
No second sentence!

Sentence Pair Classification

class label



Question Answering



Input tagging

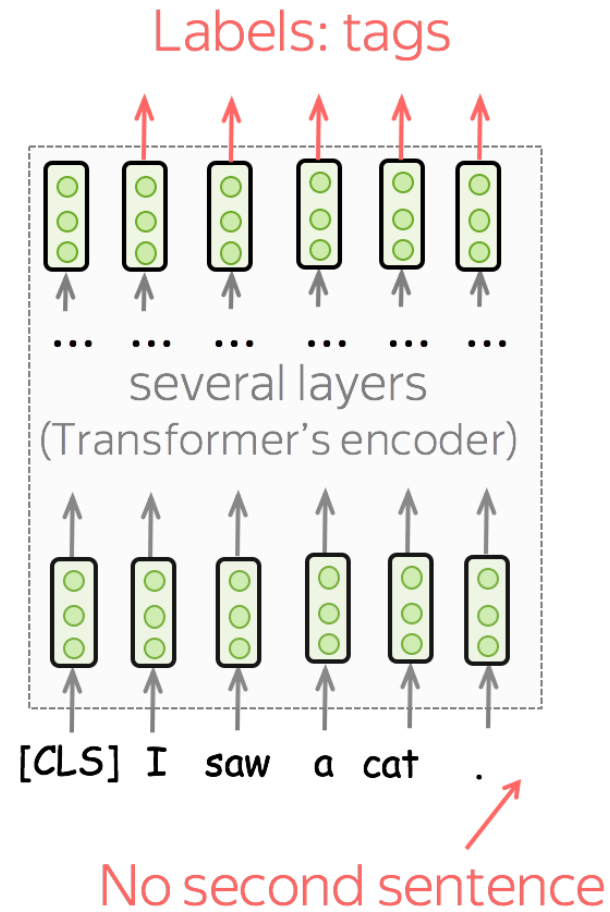
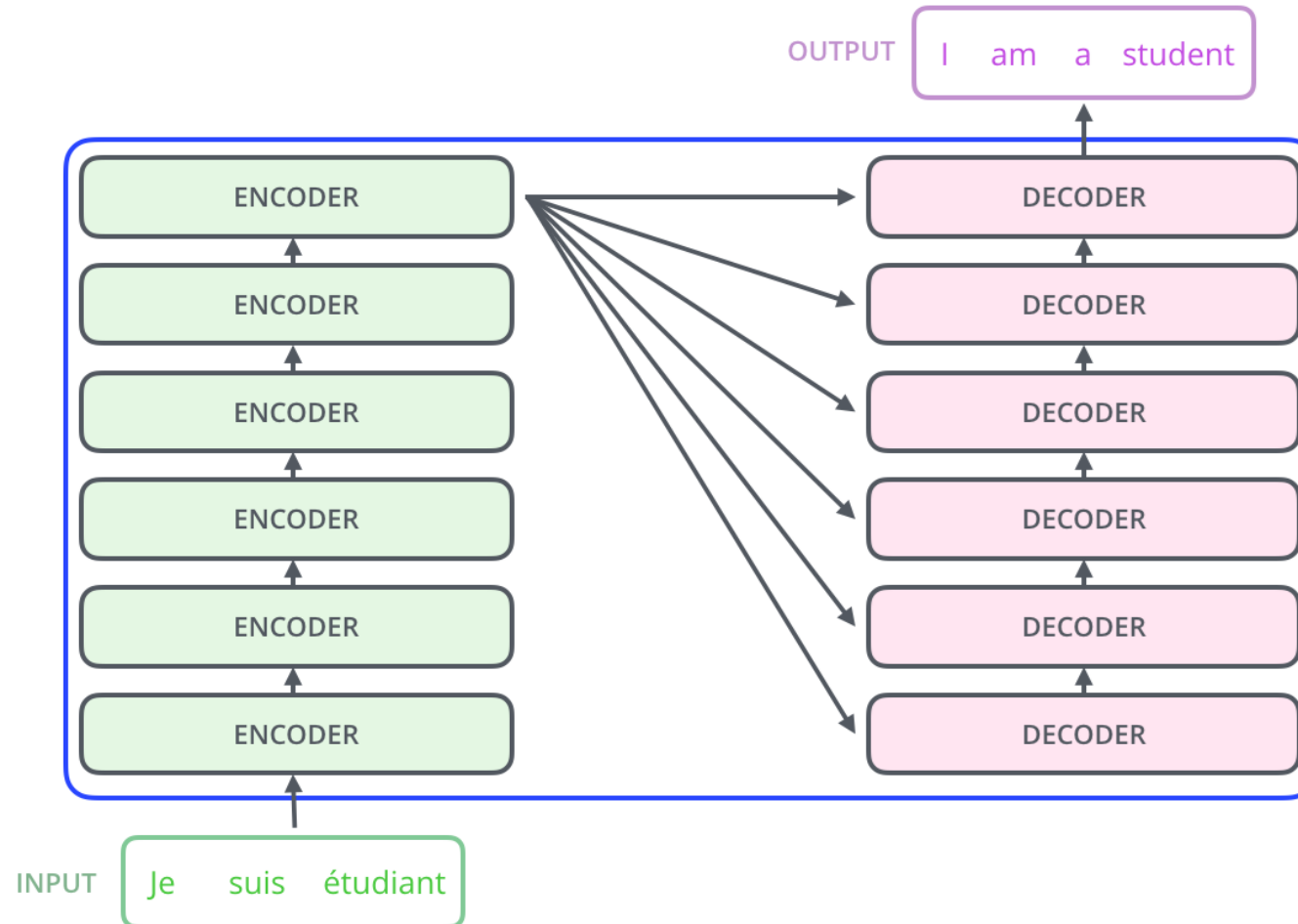




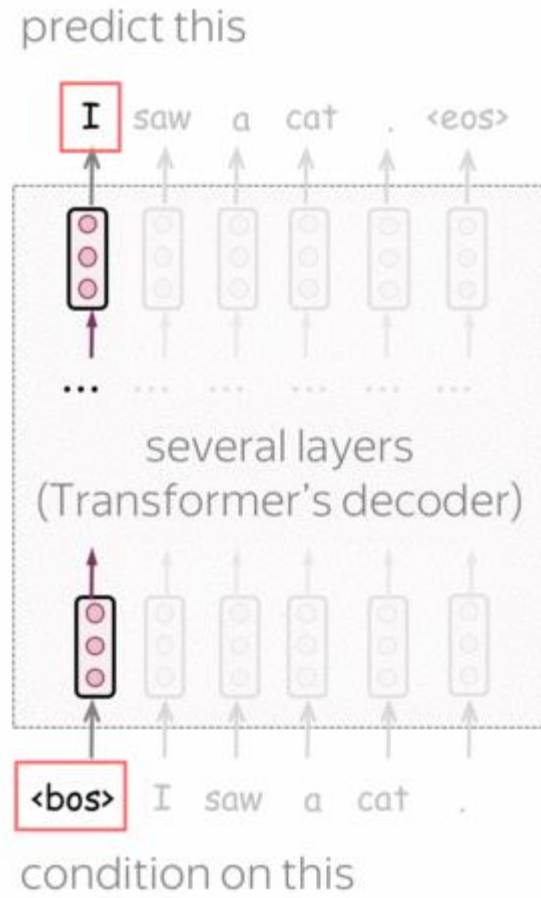
Table of Content

- The power of transfer learning
- From word-specific to contextual embeddings
- Transformer architecture overview
- BERT
- **GPT**

GPT is just a Decoder part



Decoder as a universal model



Decoder as a universal model

